

Software Engineering

Design and Implementation

Term Ambiguity: Design

In the context of **System and Software Architecture** the term “Design” refers to the **creation** of the specific System and Software **Architecture**.

In the context of **User Interfaces (UI)** and the **User Experience (UX)** of a system or software, the term “Design” refers to the **creation** of the **look and feel** and the **user workflows** of the system.

Both types of Design influence each other: A sophisticated UI and UX Design will require a specific Architectural Design while the choice of the Architectural Design will influence what UI and UX Design is possible to implement.

In this section especially, when using the term “Design”, I’m referring UI / UX Design.

In general, I’m trying to avoid the term “Design” for “System and Software Architecture” to avoid misunderstandings. It’s important for you to understand that other people might use the first definition though.

UI/UX Design

Why User Interface Design Matters

Top 10 application-design mistakes ([nn group](#))

1. **Poor feedback:** show state, help with interpretation
2. **Inconsistency:** in content (words, icons), features and navigation
3. **Error messages:** should inform about the cause
4. **Defaults:** save user effort on repetitive tasks
5. **Icons:** label icons
6. **Targets:** make clickable targets should be identifiable and reliable
7. **Modals:** do not reduce, too much context
8. **Meaningless info:** do not display long strings of alphanumerics if they are not relevant
9. **Overflow menus:** prioritize actions and throw away unused features
10. **Button proximity:** do not place destructive actions in direct contact to confirmation actions

Why User Interface Design Matters

Exercise - 15 mins - on your own:

Explore user inyerface (QR Code)
and
note down all UI mistakes that you
discover



User Inyerface

a bagaar frustration

Hi and welcome to User Inyerface,
a challenging exploration of
user interactions and design patterns.

To play the game, simply fill in the form
as fast and accurate as possible.

NO

Click HERE to GO to the next page

UI Components

- Represent the interactions between the user and a system
- Can be
 - Labels, Buttons, Tables, Lists, Views, Layouts, etc.
- Typically provided by UI frameworks
 - Large sets of standardized user interface controls
 - JavaFX, React, Flutter, etc.
 - Configuration vs. implementation
 - Reuse of functionality
 - Change of attributes (colours, sizes, etc.)
 - Change of behaviour (actions, callbacks, etc.)
 - Allow for upgradeability for core framework

UI Components: Collaboration between UI/UX Designer and Software Engineers

Pro-Tip: use a **Component Library** based on a **Design System**

Design System:

- a comprehensive set of **standards, guidelines**, and **assets** that guide the creation and consistent application of a brand's visual and experiential language **across digital products**
- Example: **Material Design**, the Design System by Google: <https://m3.material.io/>

Component Library:

- a collection of **reusable** UI components that are built **using the standards and guidelines defined in the design system**. These components are modular and can be easily combined or customized to create various parts of a user interface
- Example: **Material UI (MUI)**, a Component Library implementing Material Design for a React (a specific frontend framework) <https://mui.com/material-ui/all-components/>

Exercise - Design Studio Method - 25 to 45 mins

User Story - 5 mins

Align quickly on a user story as a team and write it down

Sketch - 10 mins

You create a design based on a user story, everyone on their own

Pitch & Critique - 10 mins

You explain your design to your team members in a quick pitch

Your team members quickly give feedback what they like and what not

Next iteration: steal and make it better (Sketch, Pitch, Critique)

One or two iterations based on the time we have left

Implementation

Implementation

Implementation is more than just coding (aka programming).

- Choice of
 - Infrastructure (Oracle or MySQL? AWS or Azure? Network topology?)
 - Frameworks (React or Angular or Vue?)
 - Libraries (to do standard operations, e.g. mathematical calculations, access to specific file types)
- Configuration of those
- Creation of development tooling
 - Automated testing
 - Automated deployment
 - Automated reporting
 - Deployment Pipeline
 - (Living) documentation

We will now focus on coding though.

Coding: Syntax and Semantics

Term from linguistics applied to coding.

Put simply, **syntax** refers to **grammar**, while **semantics** refers to **meaning**.

See <https://www.masterclass.com/articles/syntax-vs-semantics>

Syntax refers to the rules and regulations for writing any statement in a programming language

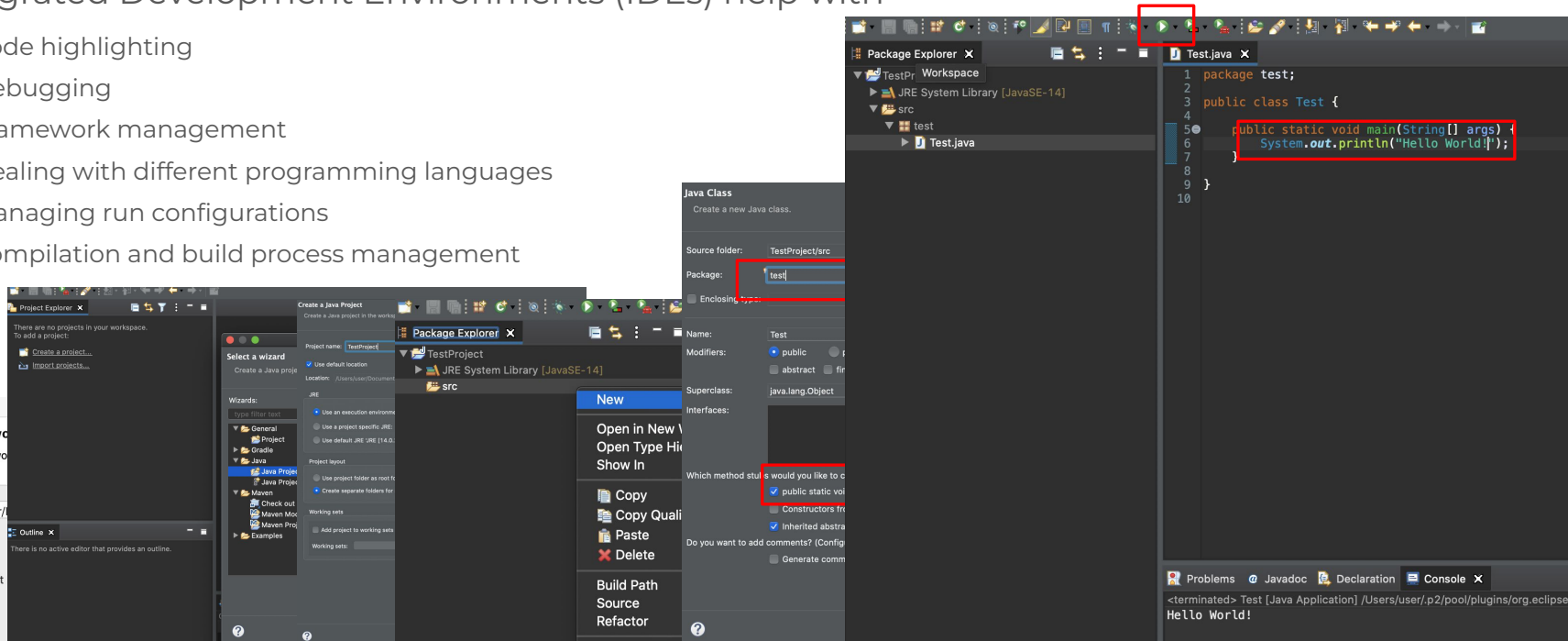
Semantics refers to the meaning associated with the statement in a programming language.

See <https://www.geeksforgeeks.org/difference-between-syntax-and-semantics/>

Compile and Type Checks help to spot **syntax errors** (code that is not executable) while **Static Code Analysis and Automated Testing** can help to spot **semantic errors** (code that makes no sense to execute).

Integrated Development Environments

- Integrated Development Environments (IDEs) help with
 - Code highlighting
 - Debugging
 - Framework management
 - Dealing with different programming languages
 - Managing run configurations
 - Compilation and build process management



Debugging

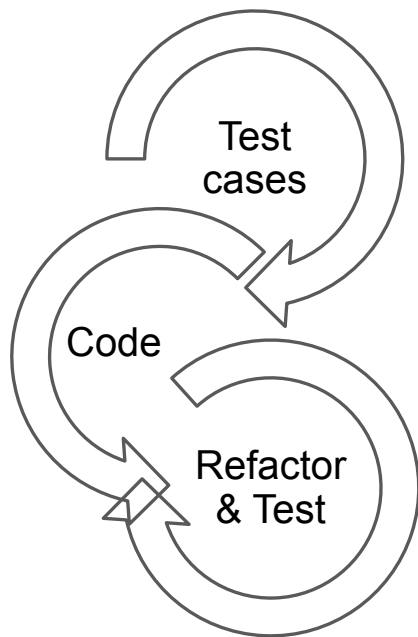
A technique to find, analyse and fix defects

Debugging is focused on the fixing of smaller defects

Debugging is part of the daily routine of developers.

Test Driven Development

TDD is a development process that **creates the tests before the solution**



1. Create a test case from the requirements
 2. Program a piece of the application so that the tests pass
 3. Refactor the code without breaking the test cases
- Repeat with the next test case

Refactoring is the process of restructuring existing code without changing its external behavior. The primary goal of refactoring is to improve the code's internal structure, making it cleaner, more efficient, and easier to understand and maintain.

Live Coding: Demonstration of TDD and Debugging

Help me find the speed that is helpful to you

- Ask questions
- Make suggestions
- Say if I should slow down or repeat

Learning to code

Coding schools

- <https://code.org/> - large variety of learning programmes
- <https://processing.org/> - Visual Framework to learn basic Java
Syntax before learning OOP

Techniques to analyse code

- Tracing / Printing
- Tests (later)
- Debugger (later)

Learning to code

AI can act as a pairing partner / teacher

- Be careful not just to copy paste - try to understand the code you use
- Turn off / ignore AI from time to time for half an hour and code without AI to check your learning progress

AI Tools

- <https://chatgpt.com/>
- <https://www.phind.com/> - focuses on coding

AI Agents

- <https://www.anthropic.com/> - Claude with Sonnet
- <https://jules.google/>
- <https://www.cursor.com/> - focuses on coding, integrated in IDE
- <https://www.jetbrains.com/help/idea/ai-assistant-in-jetbrains-ides.html> - IDE IntelliJ with AI
- <https://code.visualstudio.com/docs/copilot/overview> - IDE VSCode with AI
- <https://lmstudio.ai/> - locally installed

Building Quality in During Implementation

Software Testing Models: The Testing Pyramid

Context: Test Automation

Unit Test aka Component Test

- small piece of code tested in isolation

Service Test aka Integration Test

- two or more units tested in integration

UI Test aka End 2 End Test aka Acceptance Test

- the application tested in full integration

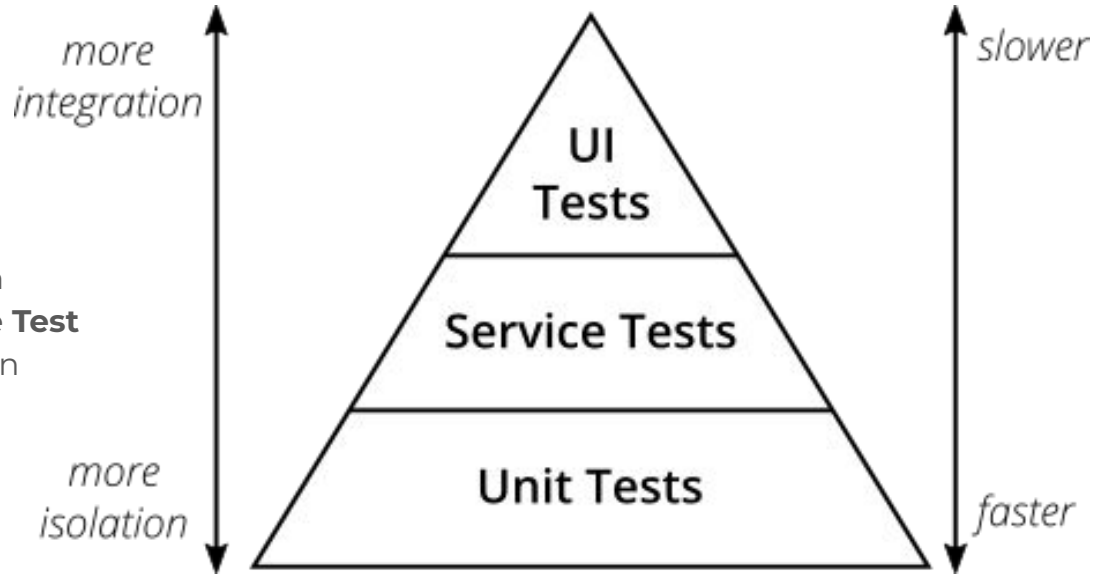
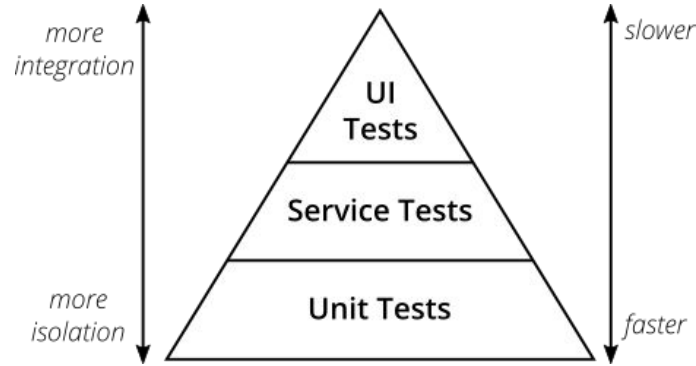


Image source: <https://martinfowler.com/articles/practical-test-pyramid.html> - originally Mike Cohn, 2009 - Further reading: [Blog post by Seb Rose: "Eviscerating the Test Automation Pyramid". 2020](#)

Software Testing Models: The Testing Pyramid

Beware of dogma: Lines are blurry, terminology may differ

- The larger the layer, the more tests you should have
- We prefer to have **more isolated tests** because they are **fast** to execute and easy to change
- **More integration** also means **higher confidence** that the system as a whole works, so we also need integrated tests even though they are slow
- The model helps you to find the right balance



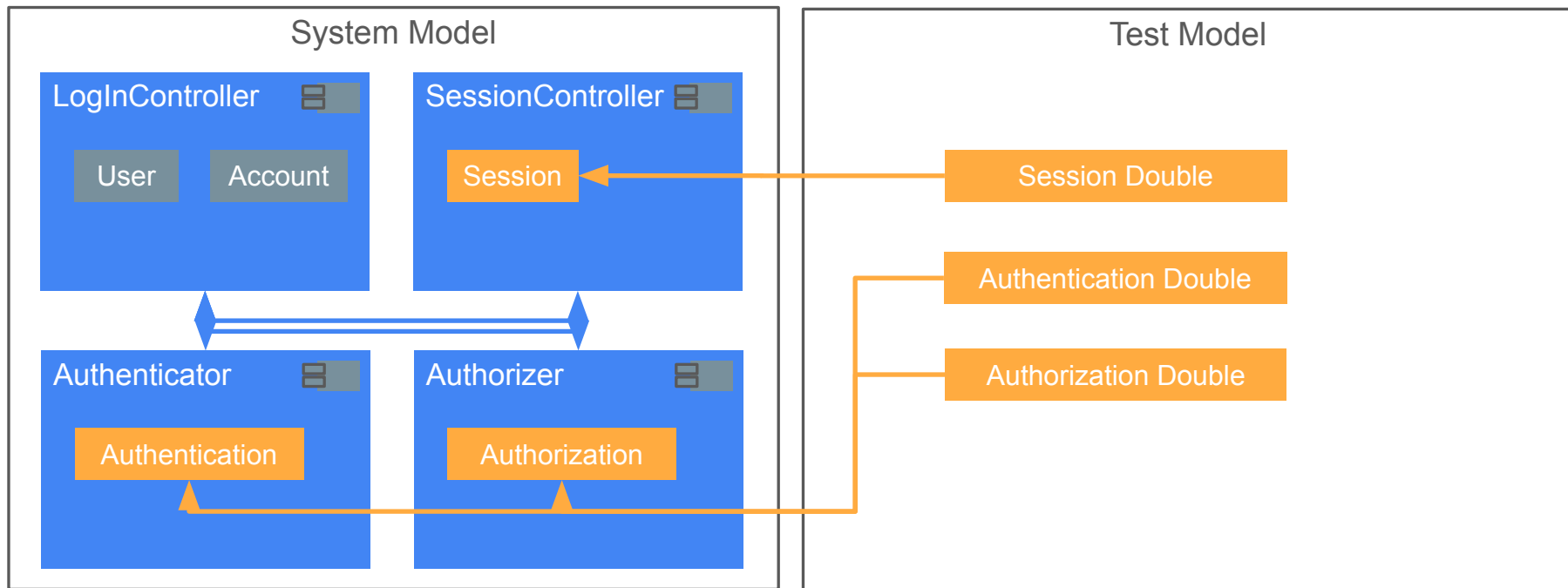
Understand the general principle: **Turning the pyramid upside down will result in high costs and increase time to market**

- Long **feedback cycles (slow tests)** delay development and release of new features
- High time invest into **maintenance of tests when changing features (high integration)** binds Software or Quality Engineers
- High time invest into **debugging to find root causes of bugs (high integration)** binds Software Engineers

Building Quality in During Implementation: Unit Testing

Unit Testing: Test Doubles

During tests we replace dependencies with test doubles



Building Quality in During Implementation: Acceptance Testing

Acceptance Testing

Acceptance tests are executed after the system is implemented and check if the **requirements** are fulfilled

Often this **involves manual testing** to actually explore the system as a user

To be sure that the functionality will not break later (**regression testing**), it is highly recommended to **automated acceptance testing**, otherwise there is continuous high manual workload that increases over time.

When automating acceptance testing, we can chose tools that create tests **via UI or via code.**

Acceptance Testing with Test First approach: BDD

Acceptance tests are executed **after the system is implemented** and check if the requirements are fulfilled.

This does not mean the tests have to be written after the system is implemented.

Writing the test scenarios **before the implementation**, enables the team to get a better understanding of the requirements and integrate the quality criterion “Testability” directly into the system.

This practise is called **BDD - Behaviour Driven Development** - and is supported by tools that link **Gherkin** to test code.

Recap: Acceptance Criteria in Gherkin

Goal: Make requirements testable

Examples of Acceptance Criteria in Gherkin (Given - When - Then)

For an upcoming conference:

Given that I'm a registered attendee

When I enter the conference venue

Then branded signs will lead me to the registration table

For a health clinic's pre-appointment paperwork:

Given that I'm an existing health center patient,

When I schedule an appointment online,

Then I will receive the fillable pre-appointment paperwork electronically

[Source: Scrum Alliance Blog](#)

Software Testing Models: The Testing Pyramid

Great place to start but there's more

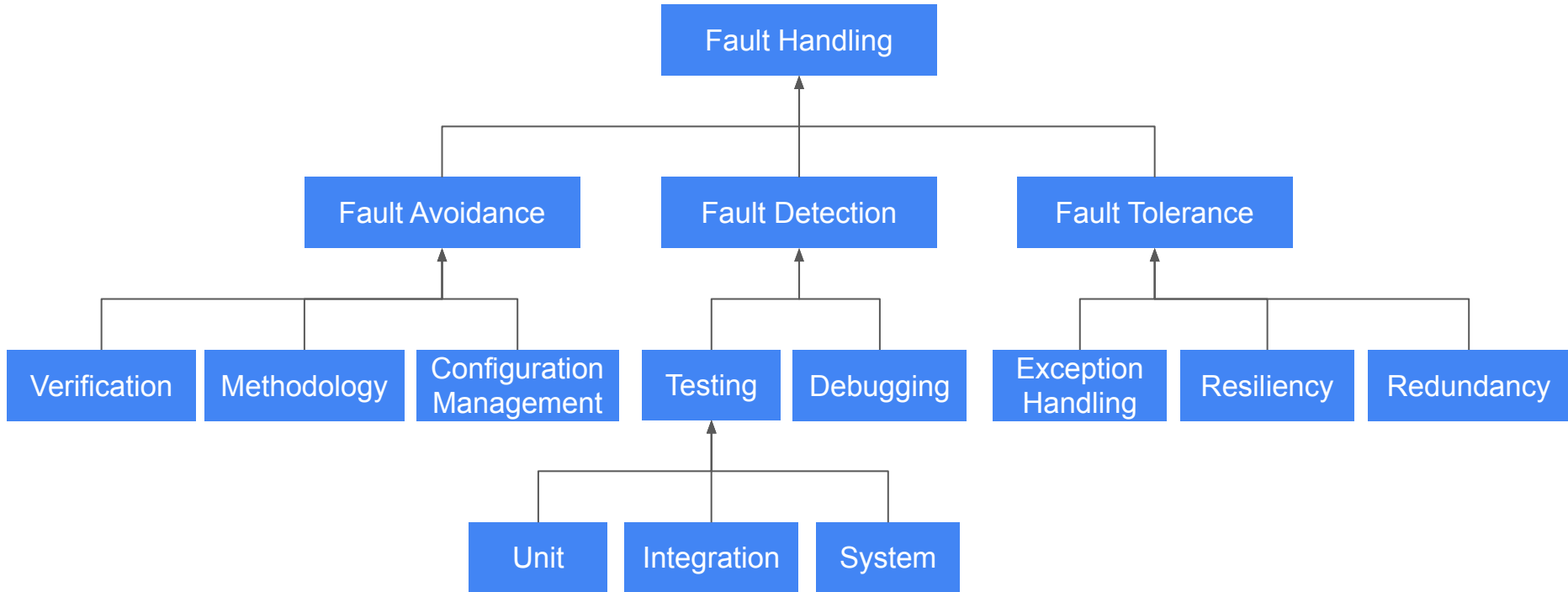
- Contract Testing
- Dependency Checks
- Static Code Analysis / Linting
- Type Checks / Compile Checks

Great Software Test Automation does not eliminate the need for Manual Testing

- Exploratory Testing
- User Testing
- But: beware of Regression Testing - this is best to be automated

Building Quality in During Implementation: There is more than just testing

Implementation: Fault Handling



Building Quality in During Implementation: 4 eye principle

4 eye principle

Fast **Feedback** (one of the agile XP values) will avoid costly rework

Applying the 4 eye principle to coding:

- Code Reviews
 - another Engineer acts as quality gateway after code is written
- Pair Programming (Pairing)
 - two Engineers work on one task on the same computer
 - one driver (coding), one navigator (guiding), switching roles from time to time
- Mob Programming (Mobbing) - now: [Ensemble Programming \(Ensembling\)](#)
 - the whole team works on one task on one computer, switching roles regularly

Engineer: Software Engineer, Quality Engineer, Infrastructure Engineer...

4 eye principle

Fast **Feedback** (one of the agile XP values) will avoid costly rework

Applying the 4 eye principle to coding:

Code Reviews

- another Engineer acts as quality gateway after code is written
- advantage: the two Engineers can manage their time independently
- disadvantage: when spotting an error the amount of rework is big
- disadvantage: people might shy away from criticizing the work
 - LGTM: looks good to me
- disadvantage: people might start to fight
 - non productive conflict
- disadvantage: code review process interrupts the flow of work
 - waiting time, waste
 - variants to counter this:
 - Synchronous Code Review: identify things to fix together - avoids ping pong
 - Non blocking Code Review: used for knowledge sharing not as quality gateway

4 eye principle

Fast **Feedback** (one of the agile XP values) will avoid costly rework

Applying the 4 eye principle to coding:

Pair Programming (Pairing)

- two Engineers work on one task on the same computer
- one driver (coding), one navigator (guiding), switching roles from time to time
- advantage: the two Engineers learn from each other
- advantage: when spotting an error, rework is easy
- advantage: when disagreeing, the two Engineers have to resolve the dispute immediately and step by step
 - productive conflict
- advantage: when one Engineer gets stuck, the other Engineer can jump in
 - better flow & fun
- disadvantage: it may feel slower. For really simple work it is. But if the Engineers do only simple work, they should better automate it anyway.
- disadvantage: requires concentration and discipline
 - take ample breaks

4 eye principle

Fast **Feedback** (one of the agile XP values) will avoid costly rework

Applying the 4 eye principle to coding:

Mob Programming (Mobbing) - now: Ensemble Programming (Ensembling)

- the whole team works on one task on one computer, switching roles regularly
- advantages: same as Pair Programming
- advantage: can leverage the expertise of multiple disciplines
 - Software, Quality, Infrastructure, even Design and Product
- disadvantage: requires a lot of concentration and discipline from the whole team
 - do not overuse
 - chose specific problems that are highly critical and need different perspectives

